SMART ROUTE FINDER USING A* SEARCH ALGORITHM

Name: Priyanshi Saxena
Registration number: 25BAI10117
Branch: CSE AI&ML
Year: 1st Year
Course Code: CSA2001
Course Title: Fundamentals In AI & ML

Submission Date: March 31, 2026

INTRODUCTION

Artificial Intelligence (AI) enables machines to mimic human intelligence and solve problems effectively. Among various AI techniques, search algorithms are fundamental for decision-making and problem-solving tasks.

In real-world scenarios such as navigation systems (like GPS), finding the shortest path between two locations is a common problem. Traditional algorithms like Breadth-First Search (BFS) or Depth-First Search (DFS) do not always provide optimal solutions efficiently.

The A* algorithm overcomes these limitations by combining:

- Actual cost from start node
- Estimated cost to goal

This project demonstrates how A* can be used to find the most efficient route between cities.

PROBLEM STATEMENT

In many real-world applications such as transportation, logistics, and navigation systems, finding the shortest or most optimal route between two points is essential.

The problem addressed in this project is:

To design and implement an intelligent system that finds the shortest path between two cities using an efficient search algorithm.

The system should:

- Accept user input for source and destination
- Compute the optimal path
- Display total cost
- Provide a visual representation

OBJECTIVES

The main objectives of this project are:

- To understand and implement the A* search algorithm
- To represent real world problems using graphs
- To apply heuristic based decision making
- To visualize the search process and results
- To demonstrate AI based problem solving techniques

METHODOLOGIES

Graph Representation

The problem is represented as a graph:

- Nodes represent cities
- Edges represent connections
- Weights represent distances

A* Algorithm

The A* algorithm uses the evaluation function:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$: Actual cost from start to node
- $h(n)$: Heuristic estimate to goal
- $f(n)$: Total cost

Working Steps

1. Initialize open list
2. Select node with lowest f(n)
3. Expand neighbours
4. Update costs
5. Repeat until goal is reached

Heuristic Function

The heuristic function estimates the remaining distance to the goal. It helps reduce unnecessary exploration and improves efficiency.

IMPLEMENTATION

Tools and Technologies

- Python
- Heapq
- NetworkX
- Matplotlib

Modules

- graph_data.py → stores graph and heuristic values
- data.py → implements A* algorithm
- visualize.py → for graph visualization

Features

- User input for start and goal
- Cost calculation (g, h and f values)
- Graph visualization

ADVANTAGES

- Faster than BFS and DFS
- Uses heuristic function for efficiency
- Guarantees optimal solution

DISADVANTAGES

- High memory usage
- Not suitable for extremely large graphs without optimization
- Not suitable for dynamic environments

RESULTS

The system successfully computes the shortest path between cities. The A* algorithm efficiently explores fewer nodes compared to uninformed search algorithms.

Example:

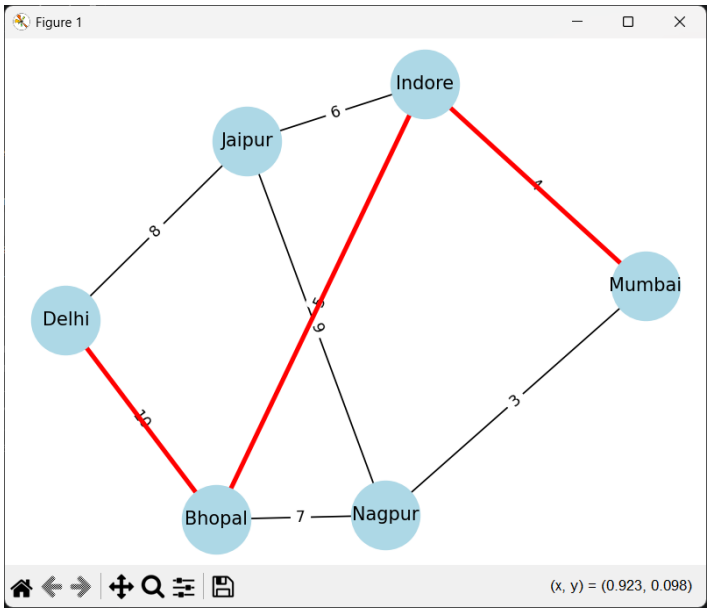
```
PS D:\Python> & C:/Users/HP/AppData/Local/Programs/Python/Python314/python.exe d:/Python/smart_route_finder/data.py
Available cities: ['Delhi', 'Bhopal', 'Jaipur', 'Indore', 'Nagpur', 'Mumbai']
Enter start city: Delhi
Enter destination city: Mumbai

Visiting: Delhi
g(n): 0, h(n): 15, f(n): 0

Visiting: Bhopal
g(n): 10, h(n): 10, f(n): 20

Visiting: Indore
g(n): 15, h(n): 5, f(n): 20

Visiting: Mumbai
g(n): 19, h(n): 0, f(n): 19
Shortest Path: Delhi → Bhopal → Indore → Mumbai
Total Cost: 19
```



CONCLUSION

This project demonstrates the application of AI search algorithms in solving real world problems. The A* algorithm provides an efficient and optimal solution for path finding tasks. By integrating visualization, the project enhances understanding and usability. The implementation highlights the importance of heuristics in improving search performance.